Sprint 3: Props, Composition, Lists, Keys, Inline Styles & Accessibility

0. Where You Are Before Sprint 3 (Sprint 3 එකට කලින් ඔයා දැනගත යුතු දේවල්)

- **Concept Explanation (Sinhala):** Sprint 3 එක පටන් ගන්න කලින්, ඔයාට කලින් Sprint වල ඉගෙන ගත්ත මේ මූලික කරුණු ගැන පැහැදිලි අදහසක් තිබිය යුතුමයි:
  - **React** කියන්නේ interactive user interfaces (UI) හදන්න පාවිච්චි කරන JavaScript library එකක්.
  - React app එකක් හදන්නේ නැවත නැවත පාවිච්චි කරන්න පුළුවන් කුඩා කොටස් වෙන **reusable components** එකතු කරලයි.
  - **Vite** කියන්නේ අපේ React project එකක් ඉක්මනින් create කරලා run කරගන්න උදව් වෙන tool එකක්.
  - අලුත් Vite React app එකක අපි සාමාන්‍යයෙන් මුලින්ම edit කරන්නේ `src/App.jsx` file එකයි.
  - **JSX** වලින් අපිට JavaScript කේතය ඇතුළේ HTML වගේ syntax එකක් ලියන්න ඉඩ ලැබෙනවා.
  - **Components** කියන්නේ JSX return කරන සාමාන්‍ය JavaScript functions වලටයි.
  - Component එකක නම හැමතිස්සෙම Capital letter එකකින් පටන් ගන්න ඕනේ (උදා: `SkillBadge`).
  - JSX වල HTML `class` වෙනුවට පාවිච්චි කරන්නේ `className` කියන keyword එකයි.
  - Component එකකින් හැමවෙලේම return කරන්න ඕනේ එකම එක parent value එකක් විතරයි. (නැත්නම් error එකක් එනවා).
  - අමතර HTML element එකක් (උදා: `<div>` එකක්) එකතු කරන්නේ නැතුව JSX elements කිහිපයක් group කරගන්න අපිට **Fragments** (`<></>`) පාවිච්චි කරන්න පුළුවන්.
  - **Default exports** import කරද්දී curly braces `{}` නැතුව import කරනවා.
  - **Named exports** import කරද්දී curly braces `{}` පාවිච්චි කරන්නම ඕනේ.
  - සාමාන්‍යයෙන් නැවත නැවත පාවිච්චි කරන UI elements `components/` කියන folder එක ඇතුළෙත්, සම්පූර්ණ screens හෝ route-level views `pages/` folder එක ඇතුළෙත් තමයි දාන්නේ.

  Problem (ප්‍රශ්නය): Sprint 2 වලදී අපි UI එක components වලට කැඩුවත්, ඒවා තාමත් තිබුණේ hardcoded (වෙනස් කරන්න බැරි) විදිහටයි.
- **Code Block:**

```
function SkillBadge() {
  return <span>React</span>;
}

export default function App() {
  return (
    <main>
      <SkillBadge />
      <SkillBadge />
      <SkillBadge />
    </main>
  );
}
```

- **Concept Explanation (Sinhala):** මෙතන තියෙන ප්‍රශ්නය තමයි හැම badge එකක් ඇතුළෙන්ම පෙන්වන්නේ "React" කියන එක විතරයි. මේක dynamic නැහැ.

1. Sprint 3 Goal (අපේ ඉලක්කය)

- **Concept Explanation (Sinhala):** Sprint 3 අවසාන වෙද්දී ඔයාට මේ වගේ components හදන්න පුළුවන් වෙන්න ඕනේ:
  - **reusable** (නැවත නැවත වෙනස් දත්ත සමඟ පාවිච්චි කළ හැකි)
  - **data-driven** (දත්ත මත පදනම්ව ක්‍රියා කරන)
  - **easier to maintain** (පහසුවෙන් නඩත්තු කළ හැකි)
  - **more accessible by default** (ඕනෑම අයෙකුට පහසුවෙන් භාවිත කළ හැකි - Accessibility)

  සරලවම කිව්වොත්, component එකක් ඇතුළට dynamic විදිහට data pass කරලා, එකම component එකෙන් වෙනස් වෙනස් UI output ගන්න එක තමයි අපේ target එක.
- **Code Block:**

```
<SkillBadge label="JavaScript" />
<SkillBadge label="React" />
<SkillBadge label="CSS" />
```

2. Sprint 3 Concepts Covered (ආවරණය වන ප්‍රධාන කරුණු)

- **Concept Explanation (Sinhala):** මේ පාඩමෙන් අපි පහත සඳහන් කරුණු 14 ගැන සම්පූර්ණයෙන්ම සාකච්ඡා කරනවා:
  1. Props
  2. Prop destructuring
  3. Props are read-only
  4. The children prop
  5. Component composition
  6. Spread props
  7. Conditional rendering
  8. Rendering lists with map()
  9. React keys
  10. Inline styles
  11. Dynamic inline styles
  12. Accessibility habits in JSX
  13. Common beginner mistakes
  14. Practice project: Team Directory

3. Props

3.1 What Are Props? (Props කියන්නේ මොනවාද?)

- **Concept Explanation (Sinhala): Props** (Properties) කියන්නේ parent component එකකින් child component එකකට pass කරන දත්ත (data) වලටයි. සරලවම හිතන්න: props කියන්නේ හරියට JavaScript function එකකට pass කරන arguments වගේ දෙයක්.
- **Code Block:**

```
// Normal JavaScript Function
function greet(name) {
  return `Hello ${name}`;
}
greet("Sangeeth");

// React Component with Props
function Greeting({ name }) {
  return <h1>Hello {name}</h1>;
}

export default function App() {
  return <Greeting name="Sangeeth" />;
}
```

- **Concept Explanation (Sinhala):** මෙතනදී `<Greeting name="Sangeeth" />` මඟින් `name` කියන prop එක pass කරනවා. `Greeting` component එක ඒක receive කරගෙන `{name}` හරහා screen එකේ display කරනවා.

3.2 Props Mental Model (Props ක්‍රියා කරන ආකාරය)

- **Concept Explanation (Sinhala):** 1. **Parent component** එක ගාව තමයි ප්‍රධාන data තියෙන්නේ. 2. Parent component එකෙන් props විදිහට data පහළට (Child එකට) pass කරනවා. 3. **Child component** එක ඒ props කියවලා (read), ඒ අනුව UI එක return කරනවා.

  රීතිය: Props හැමවෙලේම ගලාගෙන යන්නේ ඉහළ සිට පහළටයි (**Parent -> Child**). කිසිම වෙලාවක Child component එකක ඉඳන් Parent component එකකට කෙලින්ම props pass කරන්න බැහැ.

4. Why Props Are Useful (Props වැදගත් වෙන්නේ ඇයි?)

- **Concept Explanation (Sinhala):** Props නැතිව ලිව්වොත් හැම component එකකම පෙන්වන්නේ එකම hardcoded දත්තයන්. නමුත් props පාවිච්චි කළාම අපිට එකම component එක විවිධ දත්ත සමඟ නැවත නැවත පාවිච්චි කරන්න පුළුවන්.

4.1 Without Props (Props රහිතව)

- **Code Block:**

```
function SkillBadge() {
  return <span>React</span>;
}

export default function App() {
  return (
    <main>
      <SkillBadge />
      <SkillBadge />
      <SkillBadge />
    </main>
  );
}
```

- **Concept Explanation (Sinhala):** මෙතන badges 3ම පෙන්වන්නේ "React" විතරයි. මේක කිසිසේත්ම flexible නැහැ.

4.2 With Props (Props සහිතව)

- **Code Block:**

```
function SkillBadge({ label }) {
  return <span>{label}</span>;
}

export default function App() {
  return (
    <main>
      <SkillBadge label="JavaScript" />
      <SkillBadge label="React" />
      <SkillBadge label="CSS" />
    </main>
  );
}
```

- **Concept Explanation (Sinhala):** දැන් එකම `SkillBadge` component එක වෙනස් වෙනස් labels පෙන්වන්න පාවිච්චි කරන්න පුළුවන්.

4.3 Beginner Rule (ආධුනිකයින් සඳහා රීතිය)

- **Concept Explanation (Sinhala):** එකම component එකකින් වෙනස් වෙනස් data පෙන්වන්න අවශ්‍ය හැම වෙලාවකදීම **Props** පාවිච්චි කරන්න.
- **Code Block:**

```
<UserCard name="Ana" role="Frontend Developer" />
<UserCard name="Sam" role="React Learner" />

<ProductCard title="Keyboard" price={45} />
<ProductCard title="Mouse" price={20} />
```

5. Props Can Hold Different Types of Data (Props වලට දාන්න පුළුවන් Data වර්ග)

- **Concept Explanation (Sinhala):** Props විදිහට ඕනෑම JavaScript data type එකක් pass කරන්න පුළුවන්:
  - **String Prop:** කෙලින්ම quotes ඇතුළේ ලියන්න පුළුවන්. (උදා: `name="Sangeeth"`)
  - **Number Prop:** curly braces `{}` ඇතුළේ ලියන්න ඕනේ. (උදා: `price={99}`)
  - **Boolean Prop:** curly braces ඇතුළේ ලියන්න පුළුවන් (උදා: `inStock={true}`). කෙටි ක්‍රමයක් විදිහට value එකක් නොලියා prop එකේ නම විතරක් ලිව්වොත් ඒකෙන් අදහස් වෙන්නේ `true` කියන එකයි. (උදා: `<Product inStock />` කියන්නේ `inStock={true}` කියන එකටමයි).
  - **Object Prop:** Object එකක් pass කරන්නත් `{}` පාවිච්චි කරනවා.
  - **Array Prop:** Array එකක් pass කරන්නත් `{}` පාවිච්චි කරනවා.
  - **Function Prop:** Function එකක් pass කරන්නත් පුළුවන් (මේක state management සහ events වලදී පස්සේ ඉගෙන ගන්නවා).
- **Code Block:**

```
// Object Prop Example
const user = {
  name: "Ana",
  role: "Frontend Developer",
};
<UserCard user={user} />

// Array Prop Example
const skills = ["JavaScript", "React", "CSS"];
<SkillList skills={skills} />
```

6. Props Are Read-Only (Props වෙනස් කරන්න බැහැ)

- **Concept Explanation (Sinhala):** Child component එකක් ඇතුළේදී parent එකෙන් ලැබුණු props කිසිම වෙලාවක edit කරන්න හෝ mutate (වෙනස්) කරන්න බැහැ. Props කියන්නේ **Read-Only** (කියවීමට පමණක් ඇති) දේවල්.
- **Code Block:**

```
// ❌ WRONG (වැරදියි - Prop එකක් කෙලින්ම වෙනස් කරන්න හදලා)
function UserCard({ name }) {
  name = "Changed Name";
  return <h2>{name}</h2>;
}

//  CORRECT (නිවැරදියි - ලැබුණු Prop එක display කරලා විතරයි)
function UserCard({ name }) {
  return <h2>{name}</h2>;
}
```

- **Concept Explanation (Sinhala):** - **Props vs State** සරලවම මෙහෙම මතක තියාගන්න:
  - **Props:** Component එකකට පිටින් ලැබෙන දත්ත (Data a component receives).
  - **State:** Component එක ඇතුළේ වෙනස් වන දත්ත (Data a component changes).

7. Reading Props (Props කියවන ක්‍රම)

- **Concept Explanation (Sinhala):** Component එකක් ඇතුළේදී props කියවන්න ප්‍රධාන ක්‍රම 2ක් තියෙනවා:

7.1 Using the props Object (props object එක භාවිතයෙන්)

- **Concept Explanation (Sinhala):** React වලින් අපේ component function එකට එකම එක `props` object එකක් ලබා දෙනවා. අපි ඒක ඇතුළේ තියෙන values access කරන්නේ dot notation (`props.name`) එකෙන්.
- **Code Block:**

```
function UserCard(props) {
  return (
    <article>
      <h2>{props.name}</h2>
      <p>{props.role}</p>
    </article>
  );
}

// Parent pass කරන්නේ මෙහෙමයි:
<UserCard name="Ana" role="Frontend Developer" />
// ලැබෙන props object එක මේ වගේ: { name: "Ana", role: "Frontend
Developer" }
```

7.2 Destructuring Props (Props Destructure කිරීම)

- **Concept Explanation (Sinhala):** මේක වඩාත් පිරිසිදු (clean) ක්‍රමයක්. මෙතනදී අපි props object එක කෙලින්ම parameter list එක ඇතුළේදීම කඩලා ගන්නවා.
- **Code Block:**

```
function UserCard({ name, role }) {
  return (
    <article>
      <h2>{name}</h2>
      <p>{role}</p>
    </article>
  );
}
```

- **Concept Explanation (Sinhala):** `props.name` සහ `props.role` කියලා නැවත නැවත ලියනවාට වඩා, props කිහිපයක් තියෙන වෙලාවට destructuring පාවිච්චි කරන එක ලේසියි.

8. The children Prop

8.1 What Is children? (children prop එක කියන්නේ මොකක්ද?)

- **Concept Explanation (Sinhala):** `children` කියන්නේ React වල තියෙන විශේෂ built-in prop එකක්. Component එකක opening tag එක සහ closing tag එක අතර ලියන ඕනෑම JSX එකක් මේ `children` prop එකට තමයි වැටෙන්නේ.
- **Code Block:**

```
function Card({ children }) {
  return <section className="card">{children}</section>;
}

export default function App() {
  return (
    <Card>
      <h2>React</h2>
      <p>I am learning props.</p>
    </Card>
  );
}
```

- **Concept Explanation (Sinhala):** මෙතන `<Card>` tags දෙක මැද ලියපු `<h2>` සහ `<p>` tags ටික automatic විදිහට `children` prop එක හරහා `Card` component එක ඇතුළට ගිහින් display වෙනවා.

8.2 Why Is children Useful? (children වැදගත් වෙන්නේ ඇයි?)

- **Concept Explanation (Sinhala):** අපිට **Wrapper components** (වෙනත් elements වටේට යන පොදු layouts/cards) හදන්න ඕන වුණාම `children` පාවිච්චි කරනවා. පිටත layer එක (Card outline, styles) එකම වුණත් ඇතුළේ තියෙන content එක ලේසියෙන්ම වෙනස් කරන්න මේකෙන් පුළුවන්.
- **Code Block:**

```
<Card>
  <h2>Profile</h2>
  <p>Sangeeth, React learner</p>
</Card>

<Card>
  <h2>Skills</h2>
  <p>JavaScript, React, CSS</p>
</Card>
```

9. Component Composition (සංයුක්තකරණය)

9.1 What Is Composition?

- **Concept Explanation (Sinhala):** Composition කියන්නේ කුඩා components කිහිපයක් එකතු කරලා විශාල UI එකක් නිර්මාණය කිරීමේ ක්‍රියාවලියටයි. (හරියට ලෙගෝ බ්ලොක්ස් එකතු කරනවා වගේ).
- **Code Block:**

```
function Card({ children }) {
  return <section className="card">{children}</section>;
}

function UserCard({ name, role }) {
  return (
    <Card>
      <h2>{name}</h2>
      <p>{role}</p>
    </Card>
  );
}

export default function App() {
  return (
    <main>
      <UserCard name="Ana" role="Frontend Developer" />
      <UserCard name="Sam" role="React Learner" />
    </main>
  );
}
```

- **Concept Explanation (Sinhala):** - `App` component එක ඇතුළේ `UserCard` එක පාවිච්චි වෙනවා.
  - `UserCard` එක ඇතුළේ Wrapper එකක් විදිහට `Card` එක පාවිච්චි වෙනවා.
  - `Card` එක ඇතුළේ `children` හරහා dynamic elements display වෙනවා.

  මේකෙන් කේතය නැවත නැවත ලිවීම (code repetition) වැළැකෙනවා වගේම, components කුඩාවට සහ පිරිසිදුව තියාගන්න පුළුවන්.

10. Spread Props

10.1 What Are Spread Props?

- **Concept Explanation (Sinhala):** අපිට object එකක් ඇතුළේ තියෙන හැම property එකක්ම එක පාර child component එකට props විදිහට pass කරන්න අවශ්‍ය නම්, JavaScript spread operator (`...`) එක පාවිච්චි කරන්න පුළුවන්.
- **Code Block:**

```
const user = {
  id: "u1",
  name: "Ana",
  role: "Frontend Developer",
  online: true,
};

// 1. එකින් එක pass කරන සාමාන්‍ය ක්‍රමය:
<UserCard id={user.id} name={user.name} role={user.role} online=
{user.online} />

// 2. Spread Props පාවිච්චි කරලා ලියන කෙටි ක්‍රමය:
<UserCard {...user} />
```

10.2 When to Use & Avoid (භාවිතා කළ යුතු සහ නොකළ යුතු අවස්ථා)

- **Concept Explanation (Sinhala):**
  - **පාවිච්චි කරන්න:** Object එකේ තියෙන fields ටික component එකට අවශ්‍ය props වලට හරියටම සමාන නම් විතරක් පාවිච්චි කරන්න.
  - **පාවිච්චි කරන්න එපා:** Object එක ඉතාම විශාල, අදාළ නැති fields ගොඩක් තියෙන එකක් නම් spread කරන්න එපා. ඒකෙන් component එකට ලැබෙන්නේ මොනවාද කියලා තේරුම් ගන්න අමාරු වෙනවා.

11. Conditional Rendering (කොන්දේසි සහිතව UI පෙන්වීම)

11.1 What Is Conditional Rendering?

- **Concept Explanation (Sinhala):** දත්ත (data) වල තියෙන අගයන් අනුව UI එකේ විවිධ කොටස් පෙන්වීම හෝ සැඟවීම මේකෙන් අදහස් වෙනවා.
- **Code Block:**

```
function ProductCard({ title, inStock }) {
  return (
    <article>
      <h2>{title}</h2>
      {inStock ? <p>Available</p> : <p>Sold out</p>}
    </article>
  );
}
```

11.2 Three Common Ways (ප්‍රධාන ක්‍රම 3ක්)

- **Concept Explanation (Sinhala):**
  1. `if` **statements:** සාමාන්‍යයෙන් මුළු component එකෙන්ම වෙනස්ම UI එකක් return කරන්න ඕන වෙලාවට (උදා: Loading screen එකක් පෙන්වන්න).
  2. **Ternary operators (** `? :` **):** JSX එකක් ඇතුළේ කොන්දේසියක් මත පදනම්ව 'මෙන්න මේක' හෝ 'අන්න ඒක' පෙන්වන්න ඕන වුණාම (Either / Or).
  3. `&&` **operator:** කොන්දේසිය `true` නම් විතරක් යමක් පෙන්වලා, `false` නම් කිසිම දෙයක් නොපෙන්වා ඉන්න ඕන වෙලාවට (Show / Hide).

11.3 - 11.6 Method Breakdowns & Comparison

- **Code Block:**

```
// 1. if statement (ප්‍රධාන වශයෙන් විශාල UI වෙනස්කම් වලට)
function StatusMessage({ isLoggedIn }) {
  if (isLoggedIn) {
    return <p>Welcome back.</p>;
  }
  return <p>Please log in.</p>;
}

// 2. Ternary operator (JSX ඇතුළේ Options දෙකකින් එකක් තෝරන්න)
{online ? "Online" : "Offline"}

// 3. && operator (තියෙ නම් පෙන්වන්න, නැත්නම් සම්පූර්ණයෙන්ම සඟවන්න)
{isAdmin && <p>Admin tools enabled.</p>}
```

12. Rendering Lists With map() (Lists පෙන්වීම)

12.1 Why Lists Matter in React

- **Concept Explanation (Sinhala):** Web apps වල products, users, comments වගේ දේවල් පෙන්වන්න වෙන්නේ list එකක් විදිහටයි. React වලදී Array එකක් JSX බවට පත් කරන්න අපි සාමාන්‍යයෙන් පාවිච්චි කරන්නේ JavaScript වල එන `.map()` method එකයි.

12.2 Basic map() Example (සරල Array එකක් සමඟ)

- **Code Block:**

```
function SkillList() {
  const skills = ["JavaScript", "React", "CSS"];
  return (
    <ul>
      {skills.map((skill) => (
        <li key={skill}>{skill}</li>
      ))}
    </ul>
  );
}
```

12.3 Rendering a List of Objects (Objects සහිත Array එකක් සමඟ)

- **Code Block:**

```
function UserList() {
  const users = [
    { id: "u1", name: "Ana", role: "Frontend Developer" },
    { id: "u2", name: "Sam", role: "React Learner" },
  ];

  return (
    <section>
      <h2>Users</h2>
      {users.map((user) => (
        <article key={user.id}>
          <h3>{user.name}</h3>
          <p>{user.role}</p>
        </article>
      ))}
    </section>
  );
}
```

12.4 Rendering a List Using a Separate Component (වෙනම Component එකක් භාවිතයෙන්)

- **Code Block:**

```
function UserCard({ name, role }) {
  return (
    <article>
      <h3>{name}</h3>
      <p>{role}</p>
    </article>
  );
}

function UserList() {
  const users = [
    { id: "u1", name: "Ana", role: "Frontend Developer" },
    { id: "u2", name: "Sam", role: "React Learner" },
  ];

  return (
    <section>
      <h2>Users</h2>
      {users.map((user) => (
        <UserCard key={user.id} {...user} />
      ))}
    </section>
  );
}
```

13. Keys in React Lists (React වල Keys වල වැදගත්කම)

13.1 What Is a Key?

- **Concept Explanation (Sinhala):** `key` කියන්නේ list එකක තියෙන elements React වලට වෙන් වෙන්ව හඳුනා ගන්න උදව් වෙන සුවිශේෂී (unique) attribute එකක්.

13.2 Why Does React Need Keys?

- **Concept Explanation (Sinhala):** Virtual DOM එක ඇතුළේ යම් element එකක් add කරද්දී, remove කරද්දී, sort කරද්දී හෝ update කරද්දී, React එකට හරියටම ඒ element එක හඳුනාගන්න `key` එක උදව් වෙනවා. සරලවම කිව්වොත් මේක හරියට ID card number එකක් වගේ වැඩක් කරන්නේ.

13.3 - 13.6 Good Keys vs Bad Keys

- **Concept Explanation (Sinhala):**
  - **Good Keys (හොඳ ඒවා):** database එකෙන් ලැබෙන ID එක වගේ ස්ථාවර (stable) සහ වෙනස් නොවන අගයන්. (උදා: `key={user.id}`).
  - **Bad Keys (නරක ඒවා):** `Math.random()` වගේ හැම render එකකදීම අලුතින් හැදෙන අහඹු අගයන්. මේකෙන් React වල tracking අවුල් වෙලා bug ඇති වෙන්න පුළුවන්.
  - **Index Keys (array index එක key එක විදිහට පාවිච්චි කිරීම):** List එකක් filter/sort/edit වෙන්න පුළුවන් නම් කිසිම වෙලාවක index එක key එකක් විදිහට ගන්න එපා. වෙනස් නොවන static list එකකට විතරක් index එක පාවිච්චි කිරීම ප්‍රශ්නයක් නැහැ.
- **Code Block:**

```
// ❌ BAD (වැරදියි - හැම Render එකකදීම අලුත් Key එකක් හැදෙනවා)
<li key={Math.random()}>{user.name}</li>

// ⚠️ RISKY (අවදානම් සහගතයි - list එක වෙනස් වෙනවා නම් ගැටලු ඇති වෙනවා)
users.map((user, index) => <li key={index}>{user.name}</li>)

//  BEST (ඉතාම නිවැරදියි - ස්ථාවර සුවිශේෂී ID එකක්)
users.map((user) => <li key={user.id}>{user.name}</li>)
```

14. Inline Styles in React (React වල inline styles ලියන ක්‍රමය)

14.1 - 14.3 React Style Syntax vs HTML

- **Concept Explanation (Sinhala):** HTML වල style ලියන්නේ string එකක් විදිහට වුණත්, React JSX වල style එක ලියන්නේ **JavaScript Object** එකක් විදිහටයි.
- **Code Block:**

```
<!-- Normal HTML -->
<p style="color: red; font-size: 20px;">Hello</p>
```

```
// React JSX (පිටත කොළ වරහන් වලින් JavaScript කියලත්, ඇතුළත වරහන් වලින් Object එකක් කියලත් පෙන්වනවා)
<p style={{ color: "red", fontSize: "20px" }}>Hello</p>
```

14.4 - 14.5 camelCase & Style Objects

- **Concept Explanation (Sinhala):** CSS වල එන hyphenated properties (උදා: `background-color` , `font-size`) React inline styles වලදී ලියන්න ඕනේ **camelCase** වලින් (උදා: `backgroundColor` , `fontSize`).
- **Code Block:**

```
function Box() {
  const boxStyle = {
    padding: "16px",
    border: "1px solid #ccc",
    borderRadius: "8px",
    backgroundColor: "blue"
  };
  return <div style={boxStyle}>Hello</div>;
}
```

14.6 - 14.7 Dynamic Inline Styles (දත්ත අනුව වෙනස් වන inline styles)

- **Concept Explanation (Sinhala):** JavaScript දත්ත අනුව style එක වෙනස් කරන්න අපිට පුළුවන්.
- **Code Block:**

```
function StatusBadge({ online }) {
  const badgeStyle = {
    padding: "4px 8px",
    borderRadius: "8px",
    backgroundColor: online ? "lightgreen" : "lightgray",
  };

  return (
    <span style={badgeStyle}>
      {online ? "Online" : "Offline"}
    </span>
  );
}
```

15. Accessibility Habits in Sprint 3 (Accessibility පුරුදු)

15.1 - 15.3 Semantic HTML & Real Buttons

- **Concept Explanation (Sinhala):** අපේ web app එක screen reader පාවිච්චි කරන අයට වගේම keyboard එකෙන් විතරක් වැඩ කරන අයටත් පහසුවෙන් පාවිච්චි කරන්න පුළුවන් වෙන්න ඕනේ. ඒකට මේ දේවල් පුරුදු කරගන්න:
  - **Semantic HTML පාවිච්චි කරන්න:** `div` හැමතැනටම පාවිච්චි කරන්නේ නැතුව `header` , `main` , `section` , `article` , `button` වගේ අර්ථයක් තියෙන tags පාවිච්චි කරන්න.
  - **Button එකක් වෙනුවට click කරන div පාවිච්චි කරන්න එපා:** සාමාන්‍ය div එකක් keyboard එකෙන් focus කරන්න බැහැ, ඒ නිසා click කරන්න ඕන හැම තැනකටම `<button type="button">` පාවිච්චි කරන්න.
- **Code Block:**

```
// ❌ BAD (වැරදියි - div එකක් button එකක් විදිහට පාවිච්චි කරලා)
<div onClick={handleClick}>View profile</div>

//  GOOD (නිවැරදියි - නියම button element එකක්)
<button type="button" onClick={handleClick}>View profile</button>
```

15.4 Connect Labels to Inputs (Label සහ Input සම්බන්ධ කිරීම)

- **Concept Explanation (Sinhala):** input field එකකට අදාළ label එක හරියටම link කරන්න ඕනේ. HTML වල `for` වෙනුවට JSX වල පාවිච්චි කරන්නේ `htmlFor` කියන keyword එකයි. (මොකද `for` කියන්නේ JavaScript වල reserved word එකක්).
- **Code Block:**

```
<label htmlFor="email">Email</label>
<input id="email" name="email" type="email" />
```

15.5 - 15.7 Alt Text & Accessibility Checklist

- **Concept Explanation (Sinhala):**
  - අර්ථයක් තියෙන හැම image එකකටම පැහැදිලි `alt` text එකක් දෙන්න. (උදා: `alt="Team members working together"`).
  - නිකන්ම design එක ලස්සන කරන්න දාපු image එකක් නම්, alt එක හිස්ව තියන්න. (උදා: `alt=""`).
  - Keyboard එකෙන් tab key එක ඔබද්දී focus වෙන outline එක CSS වලින් `outline: none` දාලා අයින් කරන්න එපා.

16. Full Sprint 3 Example: Team Directory (සම්පූර්ණ ප්‍රායෝගික උදාහරණය)

- **Concept Explanation (Sinhala):** මේ උදාහරණය ඇතුළේ props, children, composition, conditional rendering, lists, keys, inline styles සහ accessibility කියන හැම දෙයක්ම එකට එකතු කරලා තියෙනවා.
- **Code Block:**

```
import React from 'react';

function Card({ children, featured }) {
  const cardStyle = {
    border: featured ? "2px solid gold" : "1px solid #ccc",
    padding: "16px",
    borderRadius: "12px",
    marginBottom: "12px",
  };
  return <article style={cardStyle}>{children}</article>;
}

function StatusBadge({ online }) {
  const badgeStyle = {
    padding: "4px 8px",
    borderRadius: "8px",
    backgroundColor: online ? "lightgreen" : "lightgray",
  };
  return (
    <span style={badgeStyle}>
      {online ? "Online" : "Offline"}
    </span>
  );
}

function TeamMemberCard({ name, role, online, featured }) {
  return (
    <Card featured={featured}>
      <h2>{name}</h2>
      <p>{role}</p>
      <StatusBadge online={online} />
      {featured && <p style={{ color: 'gold', fontWeight: 'bold'
}}>Featured team member</p>}
      <button type="button" style={{ marginTop: '8px', display: 'block'
}}>View profile</button>
    </Card>
  );
}

export default function App() {
  const team = [
    {
      id: "m1",
      name: "Ana",
      role: "Frontend Developer",
      online: true,
      featured: true,
    },
    {
      id: "m2",
      name: "Sam",
      role: "React Learner",
      online: false,
      featured: false,
    },
    {
      id: "m3",
      name: "Maya",
      role: "UI Designer",
      online: true,
      featured: false,
    }
  ];

  return (
    <main style={{ padding: '20px', maxWidth: '600px', margin: '0 auto'
}}>
      <h1>Team Directory</h1>
      <section aria-labelledby="team-heading">
        <h2 id="team-heading">Members</h2>
        {team.map((member) => (
          <TeamMemberCard key={member.id} {...member} />
        ))}
      </section>
    </main>
  );
}
```

17. Debug Task From Sprint 3 (කේතයේ වැරදි සෙවීම)

- **Concept Explanation (Sinhala):** මෙන්න වැරදි සහිත කේතයක් සහ එය නිවැරදි කරන ආකාරය:

17.1 Broken Code (වැරදි සහිත කේතය)

- **Code Block:**

```
function User (props) {
  return <h2>{name}</h2>;
}

const users = [{ name: "Ana" }, { name: "Sam" }]; // PDF typo fixed here

function App() {
  return users.map((user) => (
    <User key={Math.random()} user={user} style="color: red" />
  ));
}
```

17.2 - 17.4 Problems Identified (වැරදි හඳුනාගැනීම):

1. **Bad Prop Access:** `User` component එක ඇතුළේ parameter එක විදිහට `props` ගත්තට, return එකේ ලියලා තියෙන්නේ නිකන්ම `{name}` කියලා. ඒක `props.user.name` හෝ destructuring වලින් `{ user }` විදිහට ලැබෙන්න ඕනේ.
2. **Bad Key:** `key={Math.random()}` පාවිච්චි කිරීම වැරදියි. හැම render එකකදීම අලුත් key එකක් හැදෙනවා.
3. **Incorrect Inline Style Syntax:** React inline styles වලට string එකක් (`style="color: red"`) දෙන්න බැහැ, Object එකක් දිය යුතුයි.

17.5 Corrected Version (නිවැරදි කරන ලද කේතය)

- **Code Block:**

```
function User({ user }) {
  return <h2 style={{ color: "red" }}>{user.name}</h2>;
}

const users = [
  { id: "u1", name: "Ana" },
  { id: "u2", name: "Sam" },
];

export default function App() {
  return (
    <main>
      {users.map((user) => (
        <User key={user.id} user={user} />
      ))}
    </main>
  );
}
```

18. Common Sprint 3 Mistakes (ආධුනිකයින් කරන පොදු වැරදි 7ක්)

- **Concept Explanation (Sinhala):**
  1. **Using a Prop Without Receiving It:** Parameter එකක් විදිහට destructure කරලා ලබා ගන්නේ නැතුව, component එක ඇතුළේ කෙලින්ම prop එක පාවිච්චි කරන්න හැදීම.
  2. **Passing an Object but Reading the Wrong Shape:** Parent component එකෙන් pass කරන්නේ object එකක් වුණත්, child එක ඇතුළේ නිකන්ම variables විදිහට කියවන්න හැදීම. (උදා: `<User user={user} />` දීලා child එක ඇතුළේ `{name}` කියවීම).
  3. **Using Math.random() as a Key:** Virtual DOM එක අවුල් කරන නිසා කිසිම විටක key එකට `Math.random()` පාවිච්චි කරන්න එපා.
  4. **Using Index as a Key for a Changing List:** List එකක් change වෙනවා නම් array index එක key එක විදිහට පාවිච්චි කරන්න එපා.
  5. **Wrong Inline Style Syntax:** JSX ඇතුළේ HTML වගේ string එකක් විදිහට style ලියා තැබීම.
  6. **Forgetting camelCase in Inline Styles:** React style objects ඇතුළේ properties camelCase නොකර CSS විදිහටම ලියන්න හැදීම (උදා: `fontSize` වෙනුවට `"font-size"` ලිවීම).
  7. **Using Clickable divs Instead of Buttons:** Accessibility නීති කඩ කරමින් button එකක් වෙනුවට onClick දාපු div එකක් පාවිච්චි කිරීම.

19. Sprint 3 Practice Project: Product List

- **Concept Explanation (Sinhala):** මේක dynamic product list එකක් හදන සරල practice project එකක්. මෙතනදී Products array එක `.map()` එකෙන් render කරලා, `ProductCard` එකට spread props හරහා දත්ත යවනවා.
- **Code Block:**

```
import React from 'react';

function ProductCard({ title, price, inStock }) {
  return (
    <article style={{ border: '1px solid #ddd', padding: '10px', margin:
'10px 0', borderRadius: '5px' }}>
      <h2>{title}</h2>
      <p>${price}</p>
      {inStock ? (
        <button type="button">Buy</button>
      ) : (
        <p style={{ color: 'red' }}>Sold out</p>
      )}
    </article>
  );
}

function ProductList() {
  const products = [
    { id: "p1", title: "Keyboard", price: 45, inStock: true },
    { id: "p2", title: "Mouse", price: 20, inStock: false },
    { id: "p3", title: "Monitor", price: 180, inStock: true },
  ];

  return (
    <section>
      <h2>Products</h2>
      {products.map((product) => (
        <ProductCard key={product.id} {...product} />
      ))}
    </section>
  );
}

export default function App() {
  return (
    <main style={{ padding: '20px' }}>
      <h1>Shop</h1>
      <ProductList />
    </main>
  );
}
```

20. Sprint 3 Practice Project: Team Directory

- **Concept Explanation (Sinhala):** (මෙම ව්‍යාපෘතිය ඉහත 16 වන කොටසේ සම්පූර්ණයෙන්ම සවිස්තරාත්මකව කේතනය කර ඇති බැවින්, එය නැවත අධ්‍යයනය කර පුහුණු වන්න.)

21. Beginner+ Practice Extension (Accessibility Audit Checklist)

- **Concept Explanation (Sinhala):** ඔයා හදපු project එකක accessibility තත්ත්වය පරීක්ෂා කරන්න මේ checklist එක පාවිච්චි කරන්න:
  - [ ] ප්‍රධාන UI එක වටේට `<main>` element එකක් තියෙනවාද?
  - [ ] හැම section එකකටම පැහැදිලි headings (`h2` , `h3` වගේ) තියෙනවාද?
  - [ ] Cards හෝ single items පෙන්වන්න `<article>` tag එක පාවිච්චි කරලා තියෙනවාද?
  - [ ] Interactive actions (click කරන තැන්) වලට නියම `<button>` elements ම පාවිච්චි කරලා තියෙනවාද?
  - [ ] form inputs වලට `htmlFor` හරහා ලේබල් සම්බන්ධ කරලාද තියෙන්නේ?
  - [ ] හැම image එකකටම සුදුසු `alt` text එකක් තියෙනවාද?
  - [ ] Map කරන හැම list item එකකටම stable key එකක් දීලා තියෙනවාද? (කිසිම තැනක `Math.random()` නැද්ද?)

22. Sprint 3 Revision Questions and Answers (ප්‍රශ්නෝත්තර)

- **Concept Explanation (Sinhala):**
  - **ප්‍රශ්නය 1: reusable components සඳහා props වැදගත් වෙන්නේ ඇයි?**
    - **පිළිතුර:** Props මඟින් එකම component එකට වෙනස් වෙනස් දත්ත ලබා දීලා, විවිධ තැන් වලදී විවිධ අයුරින් නැවත භාවිත කරන්න ඉඩ සලසනවා.
  - **ප්‍රශ්නය 2:** `children` **prop එක ඇතුළේ තියෙන්නේ මොනවාද?**
    - **පිළිතුර:** Component එකක opening tag සහ closing tag අතර අපි ලියන ඕනෑම JSX content එකක් `children` prop එක ඇතුළේ තියෙනවා.
  - **ප්‍රශ්නය 3: array index එක key එක විදිහට පාවිච්චි කිරීම අවදානම් වෙන්නේ ඇයි?**
    - **පිළිතුර:** List එකේ items filter, sort, add හෝ delete වෙද්දී array index එක එකම අගයක පවතින්නේ නැහැ. React එකට elements මාරු වුණාම පැටලෙන්න පුළුවන් නිසා index එක key එකක් විදිහට පාවිච්චි කිරීම අවදානම් සහිතයි.
  - **ප්‍රශ්නය 4:** `style={{ color: "red" }}` **කේතයේ curly braces ජෝඩු දෙකක් තියෙන්නේ ඇයි?**
    - **පිළිතුර:** පළමු curly braces ජෝඩුවෙන් කියන්නේ "JSX ඇතුළේ JavaScript ලියනවා" කියන එකයි. දෙවන curly braces ජෝඩුවෙන් කියන්නේ "මේ ලියන්නේ JavaScript Object එකක්" කියන එකයි.
  - **ප්‍රශ්නය 5: div එකකට onClick දානවාට වඩා button එකක් පාවිච්චි කිරීම හොඳ ඇයි?**
    - **පිළිතුර:** Button එකක් පාවිච්චි කළාම keyboard focus එක, keyboard එකෙන් activate (Enter/Space ඔබලා ක්‍රියාත්මක) කිරීමේ හැකියාව සහ screen reader සහයෝගය automatic ලැබෙනවා. Div එකකට ඒ කිසිවක් ලැබෙන්නේ නැහැ.

23. Sprint 3 Self-Check Questions (ස්වයං ඇගයීම් ප්‍රශ්න)

- **Concept Explanation (Sinhala):** ඔයාට Sprint 4 එකට යන්න පුළුවන්ද කියලා තීරණය කරන්න මේ ප්‍රශ්න 20 ට සටහන් බලන්නේ නැතුව උත්තර දෙන්න පුළුවන්ද කියලා බලන්න:
  1. What are props?
  2. Why are props useful?
  3. What does children contain?
  4. What does `{...user}` do?
  5. Why should props not be changed by a child component?
  6. What is component composition?
  7. When should you use an if statement for rendering?
  8. When should you use a ternary?
  9. When should you use `&&` ?
  10. How does `map()` help render lists?
  11. Why does a list need a key?
  12. Why is `Math.random()` a bad key?
  13. When is an index key risky?
  14. Why does React use `style={{ color: "red" }}` ?
  15. Why do CSS property names become camelCase in inline styles?
  16. Why is a real button better than a clickable div?
  17. How do you connect a label to an input in JSX?
  18. When should an image have meaningful alt text?
  19. When should an image use `alt=""` ?
  20. What should your Team Directory project prove?

24. Sprint 3 Mini Revision Sheet (කෙටි පුනරීක්ෂණය)

- **Concept Explanation (Sinhala):**
  - **Props:** Parent -> Child දත්ත ගලා යාම. `<Greeting name="Sangeeth" />`
  - **Destructuring:** Function argument එකක් විදිහටම variables කඩලා ගැනීම. `function Greeting({ name })`
  - **Read-Only:** Props කිසිවිටක වෙනස් නොකළ යුතුය.
  - **children:** Tags දෙක මැද ඇති JSX අල්ලා ගැනීම. `<Card><p>Hello</p></Card>`
  - **Composition:** කුඩා components එකතු කර විශාල UI සෑදීම.
  - **Spread Props:** Object එකක properties සියල්ල එකවර pass කිරීම. `<UserCard {...user} />`
  - **Conditional Rendering:** `online ? "Online" : "Offline"` සහ `featured && <p>Featured</p>`
  - **Lists:** map() මඟින් array එකක් JSX කිරීම. `users.map(u => <Card key={u.id} {...u} />)`
  - **Keys:** `key={user.id}` (ස්ථාවර සුවිශේෂී අගයක් විය යුතුය).
  - **Inline Styles:** Object එකක් විය යුතුය. `style={{ color: "red", fontSize: "20px" }}`
  - **Accessibility:** Semantic elements, real buttons, `htmlFor` සහ meaningful alt text භාවිතය.

25. Final Beginner Explanation & Summary

- **Concept Explanation (Sinhala):** සරලවම කිව්වොත්: "මට React components නැවත නැවත වෙනස් දත්ත සමඟ පාවිච්චි කරන්න **props** උදව් වෙනවා. Wrapper components හදන්න මට **children** පාවිච්චි කරන්න පුළුවන්. Components එකක් ඇතුළේ තවත් එකක් දාලා මට **composition** කරන්න පුළුවන්. `if` , ternary (`? :`) සහ `&&` පාවිච්චි කරලා දත්ත අනුව UI එක වෙනස් කරන්න පුළුවන්. Array එකක් UI එකේ පෙන්වන්න මට `map()` පාවිච්චි කරන්න පුළුවන් වගේම හැම list item එකකටම වෙනස් නොවන **key** එකක් දෙන්නත් ඕනේ. දත්ත මත පදනම්ව style කරන්න මට inline style objects පාවිච්චි කරන්න පුළුවන්. හැම වෙලාවකදීම UI එක හදද්දී accessibility Habits (semantic tags, real buttons, labels) පිළිපදින්න මම වග බලා ගන්නවා."